Universidad Nacional de Colombia
Facultad de Ingeniería



PROTOTIPO 1

NetPulse: requisitos, diseño y API segura sobre Odoo

Presentado por:

Juliana Parra Caro
Manuel Federico Castro Suarez
Fabian David Mora Martinez
Andres Felipe Arias Gonzalez

Asignatura:
Ingeniería de Software II

Semestre 2026-II
Facultad de Ingeniería

1. Requisitos **3**
1.1 Visión del producto **3**
1.2 Roles de usuario y permisos **3**
1.3 Historias de usuario y criterios de aceptación **3**
1.4 Story map y MVP **6**
1.5 Estimación (Planning Poker) y tablero **6**
2. Experiencia de usuario **7**
2.1 Wireframes **7**
2.2 Flujo de navegación **7**
2.3 Modelo de cliente **8**
3. Diseño de la solución **9**
3.1 Arquitectura **9**
3.2 Modelo de dominio (DDD) **9**
Lenguaje ubicuo (glosario) **9**
Bounded contexts y context map **10**
Agregados **11**
3.3 Diagrama de clases y SOLID **12**
3.4 Patrones de diseño (GoF) **13**
3.5 Atributos de calidad **13**
4. API segura (parte implementada) **14**
4.1 Especificación OpenAPI **14**
4.2 Backend y organización por capas **15**
4.3 Autenticación **16**
4.4 Autorización por roles (401 vs. 403) **16**
4.5 Buenas prácticas de seguridad **16**
4.6 HTTPS **17**
4.7 Colección de Postman **17**
5. Decisiones, riesgos y próximos pasos **17**
5.1 Decisiones clave **17**
5.2 Riesgos **18**
5.3 Próximos pasos hacia el Prototipo 2 **18**
Anexo A. Cobertura de la guía del Prototipo 1 **18**

1. Requisitos

1.1 Visión del producto

Los equipos de redes de una organización detectan las caídas de dispositivos de forma manual o por avisos informales, y las incidencias se registran en otra herramienta, a mano y sin priorizar. Eso retrasa la reacción cuando cae un equipo crítico (un switch troncal, por ejemplo). **NetPulse** resuelve este problema para administradores y técnicos de redes: monitorea el estado de los dispositivos (GNS3 en el laboratorio) y, cuando un dispositivo crítico cae, crea automáticamente una incidencia en Odoo, evita duplicados, permite asignarla a un técnico y controla su ciclo de vida (Pendiente → En proceso → Resuelta).

Enfoque low-code. NetPulse se construye como un módulo de Odoo 17 Community. Odoo aporta autenticación, usuarios y grupos, persistencia, interfaz web responsiva, historial de cambios y una API externa; el equipo escribió la lógica de dominio (modelos, reglas de negocio, integración con GNS3), las vistas y los permisos. Algunos requisitos de la guía están pensados para backends escritos desde cero (JWT, endpoints REST propios); en esos casos se documenta el equivalente en Odoo y la diferencia. El Anexo A resume la cobertura de la guía.

1.2 Roles de usuario y permisos

Rol	Puede
Administrador	Todo: acceso completo a dashboard y topología, CRUD de dispositivos e incidencias, asignar técnico y prioridad, simular caídas y sincronizar con GNS3.
Técnico	Ver dispositivos; ver, crear y atender solo sus incidencias (iniciar atención, marcar resuelta). No elimina dispositivos, no asigna técnico ni cambia prioridad.
Sistema de monitoreo (actor no humano)	Acción programada de Odoo (cada minuto) o script externo que informa el estado de cada nodo; dispara la creación automática de incidencias.

1.3 Historias de usuario y criterios de aceptación

HU-01 Iniciar sesión. Como usuario (administrador o técnico) quiero iniciar sesión para acceder a las funciones de mi rol.

> Dado que tengo credenciales válidas, cuando inicio sesión en Odoo, entonces entro a la aplicación NetPulse y veo solo los menús de mi rol (el técnico no ve Dashboard, Topología ni Configuración).
>
> Dado que escribo una contraseña incorrecta, cuando inicio sesión, entonces Odoo rechaza el acceso y no abre sesión; por API, `authenticate` devuelve `false` y `execute_kw` responde `AccessDenied` (≈401).

HU-02 Gestionar usuarios y roles. Como administrador quiero crear usuarios y asignarles el rol Administrador o Técnico para controlar quién puede hacer qué. (No se implementó registro público: las cuentas las crea el administrador.)

Dado que creo un usuario con el grupo Técnico, cuando inicia sesión, entonces solo ve las opciones del técnico.

Dado que soy técnico, cuando intento abrir Ajustes → Usuarios, entonces no tengo acceso a esa opción.

HU-03 Ver la topología y el estado de la red. Como administrador quiero ver los dispositivos activos (verde) y caídos (rojo) para conocer el estado de la red de un vistazo.

Dado que hay dispositivos con estado conocido, cuando abro Dashboard o Topología, entonces los activos aparecen en tarjetas verdes y los caídos en rojas, y el dashboard muestra el número de dispositivos por estado (Caído, Activo, Desconocido).

Dado que GNS3 no responda, cuando corre la sincronización, entonces los dispositivos asociados a un nodo pasan a Desconocido y se muestran sin color de alerta.

HU-04 Crear incidencia automática. Como administrador quiero que al caer un dispositivo crítico se cree automáticamente una incidencia sin revisar la red a mano.

Dado un dispositivo crítico activo, cuando el monitoreo lo detecta caído, entonces se crea una incidencia en estado Pendiente, prioridad Alta y causa raíz «<dispositivo> offline».

Dado un dispositivo no crítico, cuando cae, entonces cambia a Caído pero no se crea incidencia automática.

HU-05 No duplicar incidencias. Como administrador quiero que una caída repetida no genere otra incidencia para no saturar la bandeja.

Dado un dispositivo con una incidencia abierta, cuando se detecta de nuevo la caída, entonces no se crea una incidencia nueva.

Dado que la incidencia anterior ya está Resuelta, cuando el dispositivo vuelve a caer, entonces sí se crea una incidencia nueva.

HU-06 Gestionar dispositivos. Como administrador quiero crear, consultar, editar y eliminar dispositivos para mantener el inventario monitoreado.

Dado que soy administrador, cuando creo un dispositivo con nombre y nodo GNS3, entonces queda en el inventario y aparece en la topología.

Dado que soy técnico, cuando intento eliminar un dispositivo, entonces el sistema responde `AccessError` (≈403) y el dispositivo no se elimina.

HU-07 Asignar técnico y prioridad. Como administrador quiero asignar una incidencia a un técnico y fijar su prioridad para repartir el trabajo.

Dado una incidencia Pendiente sin técnico, cuando la asigno, entonces queda a nombre del técnico y este recibe una actividad pendiente.

Dado que soy técnico, cuando intento asignar la incidencia o cambiar su prioridad, entonces el sistema responde `AccessError` (≈403).

HU-08 Ver solo mis incidencias. Como técnico quiero ver solo las incidencias asignadas a mí para concentrarme en mi trabajo.

Dado que existen tres incidencias y solo una es mía, cuando listo incidencias, entonces veo únicamente la mía.

Dado que consulto por identificador una incidencia ajena, cuando la pido, entonces la búsqueda no la devuelve y la lectura directa responde `AccessError` (regla de acceso por registro).

HU-09 Atender y resolver. Como técnico quiero iniciar la atención y marcar como Resuelta mi incidencia para reportar el avance.

Dado una incidencia Pendiente asignada a mí, cuando inicio la atención y luego la resuelvo, entonces pasa a En proceso y después a Resuelta (con fecha de cierre).

Dado una incidencia Pendiente, cuando intento marcarla Resuelta sin iniciarla, entonces el sistema responde `ValidationError` (≈400) y el estado no cambia.

HU-10 Sincronizar con GNS3. Como administrador quiero que el sistema consulte GNS3 cada minuto para detectar caídas.

Dado un nodo detenido en GNS3, cuando corre la sincronización, entonces el dispositivo pasa a Caído en ≤ 2 minutos.

Dado que GNS3 no responde, cuando corre la sincronización, entonces los estados pasan a Desconocido y no se crean incidencias falsas.

HU-11 Simular caída y recuperación. Como administrador quiero simular la caída y la recuperación de un dispositivo para demostrar el flujo sin depender de GNS3. (Si GNS3 está configurado, el siguiente ciclo del monitor puede sobrescribir el estado simulado.)

Dado que soy administrador, cuando pulso «Simular caída» en un dispositivo crítico (por ejemplo SW2), entonces el dispositivo pasa a Caído y se crea la incidencia como en HU-04.

Dado que soy técnico, cuando intento usar la simulación, entonces el botón no está disponible y, por API, el sistema responde `AccessError` (≈403).

HU-12 Consumir la API documentada. Como integrador quiero una especificación OpenAPI para construir clientes sin leer el código.

Dado el archivo `openapi.yaml`, cuando lo abro en Swagger Editor, entonces no muestra errores y documenta las operaciones del MVP con esquemas y ejemplos.

Dado que llamo a la API con una API key inválida, cuando hago la petición, entonces recibo un error `AccessDenied` (≈401) en el cuerpo de la respuesta y no se ejecuta la operación.

1.4 Story map y MVP

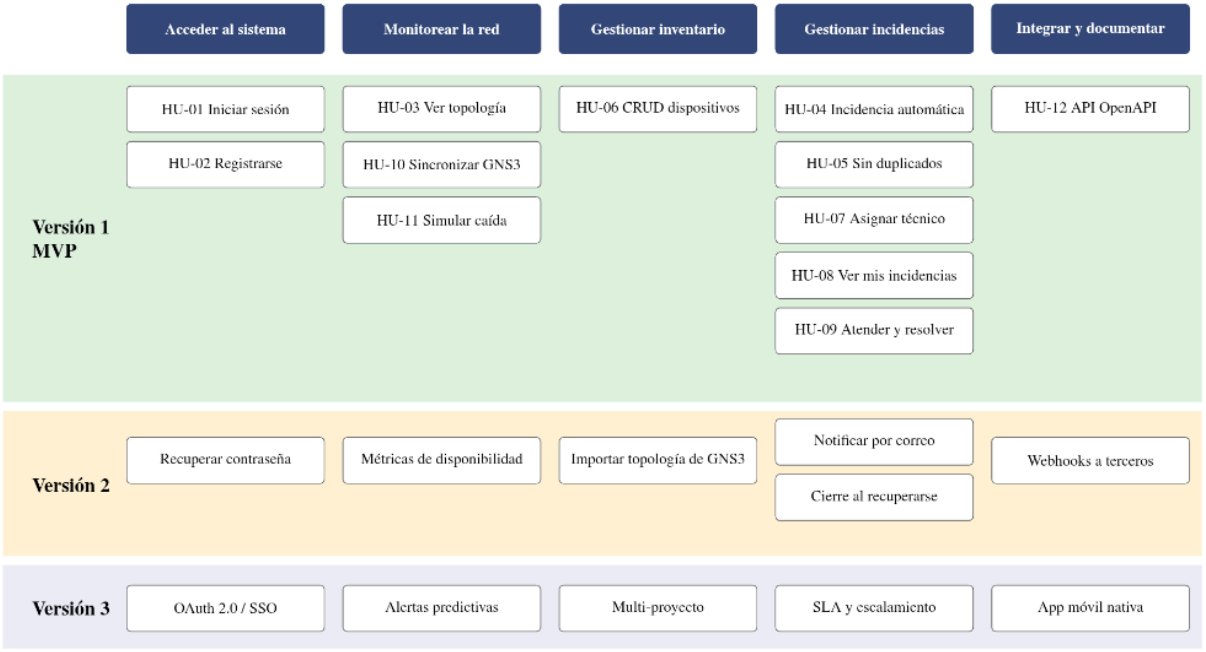


Figura 1: Story map de NetPulse. Las historias de la Versión 1 (HU-01 a HU-12) forman el MVP.

1.5 Estimación (Planning Poker) y tablero

Estimamos con Planning Poker, serie de Fibonacci (1, 2, 3, 5, 8, 13), tomando como historia de referencia HU-11 (2 puntos: dos botones y un cambio de estado). Cada integrante votó en privado y se discutieron las diferencias grandes (p. ej., HU-04 y HU-10 subieron a 8 por integrar GNS3 y evitar duplicados).

HU	Historia	Puntos	Por qué
HU-01	Iniciar sesión	3	Emisión de JWT y validación de credenciales
HU-02	Registrarse	3	Validaciones y hash de contraseña
HU-03	Ver topología/estado	5	Vista gráfica con colores por estado
HU-04	Incidencia automática	8	Regla de negocio, prioridad y causa raíz
HU-05	Sin duplicados	3	Consulta de incidencia abierta por dispositivo
HU-06	CRUD dispositivos	5	Cuatro operaciones con validaciones y roles
HU-07	Asignar técnico/prioridad	5	Permisos, actividad pendiente
HU-08	Ver mis incidencias	3	Reglas de acceso por registro
HU-09	Atender y resolver	5	Máquina de estados
HU-10	Sincronizar con GNS3	8	Adaptador externo, tiempos de espera, estado Desconocido
HU-11	Simular caída	2	Referencia
HU-12	API OpenAPI	3	Redacción y validación del YAML
	Total MVP	**53**	

El tablero de GitHub Projects tiene las columnas Backlog, Listo, En progreso, En revisión (pull request) y Hecho, con las 12 historias del MVP, su estimación y una rama `feature/hu-XX-nombre` por historia.

2. Experiencia de usuario

2.1 Wireframes

Los wireframes de algunas pantallas del MVP, en versión web, están en Figma: https://www.figma.com/design/Z7XpwDRNMefjx80Mwmpafs/Wireframe-NetPulse?node-id=0-1&t=L85pH1zrQ5Nr9IPm-1

Los wireframes son el diseño de referencia (paleta y tipografía del wireframe oficial de Odoo). La implementación usa las vistas nativas de Odoo (kanban, lista, formulario y gráfico), por lo que el aspecto final no es idéntico. No se elaboraron wireframes móviles: el cliente web de Odoo es responsivo y se adapta al navegador del celular.

Pantalla	Rol	HU	Qué muestra
Inicio de sesión	Todos	HU-01	Pantalla de acceso de Odoo: usuario y contraseña
Dashboard / topología	Administrador	HU-03	Tarjetas de dispositivos en verde, rojo o neutro (Desconocido) y número de dispositivos por estado
Bandeja de incidencias	Administrador	HU-04, 05, 07	Todas las incidencias, filtros por estado y prioridad
Formulario de dispositivo	Administrador	HU-06, 11	Nombre, tipo, criticidad, ID de nodo GNS3; botones «Simular caída» y «Simular recuperación»
Detalle de incidencia	Administrador y técnico	HU-07, 08, 09	Causa raíz y dispositivo en la parte superior; botones «Iniciar atención» y «Marcar resuelta»

2.2 Flujo de navegación

El flujo representa las acciones principales de cada tipo de usuario, desde el inicio de sesión hasta su tarea principal.

Administrador:

1. Inicio de sesión
2. Dashboard / topología
3. Lista de dispositivos
4. Bandeja de incidencias
5. Detalle: asignar técnico y prioridad

Técnico:

1. Inicio de sesión
2. Mis incidencias
3. Detalle de incidencia
4. Iniciar atención
5. Marcar resuelta

2.3 Modelo de cliente

Elegimos una **aplicación web responsiva**: el cliente web de Odoo, servido por el mismo backend y accesible desde el computador o el navegador del celular. No se desarrolló un cliente móvil aparte ni una PWA instalable; esta última queda como trabajo futuro.

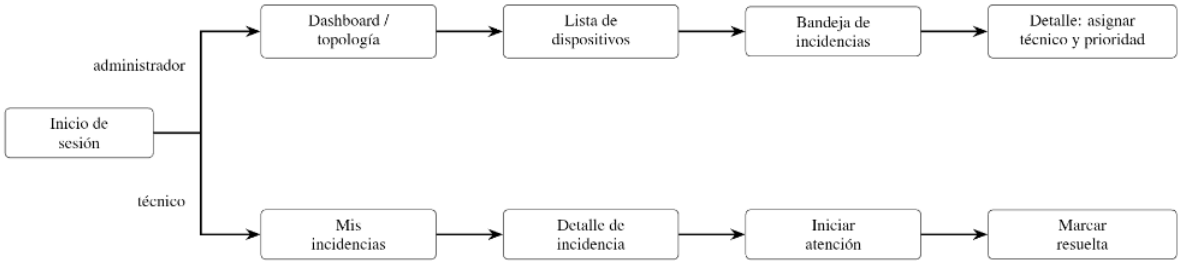


Figura 3: Flujo desde el inicio de sesión hasta la tarea principal de cada rol.

Criterio	**Web responsiva (Odoo)**	**Híbrido**	**Multiplataforma**	**Nativo**
Costo y tiempo	Bajo: ya existe con Odoo	Medio	Medio-alto	Alto
Reutiliza el backend	Total	Parcial	Parcial	Parcial
Distribución	Un enlace, sin tienda	Tiendas	Tiendas	Tiendas
Hardware del dispositivo	No lo necesitamos	Sí	Sí	Sí
Notificaciones push	Limitadas	Sí	Sí	Sí
Habilidades del equipo	Python/XML (ya las tenemos)	Nueva pila	Nueva pila	Nueva pila

NetPulse no necesita sensores ni cámara, el técnico solo consulta y cambia estados, y el equipo ya domina Odoo; por eso el cliente web minimiza costo y riesgo. Si en prototipos posteriores hacen falta notificaciones push nativas, un cliente híbrido podría consumir la misma API externa de Odoo.

3. Diseño de la solución

3.1 Arquitectura

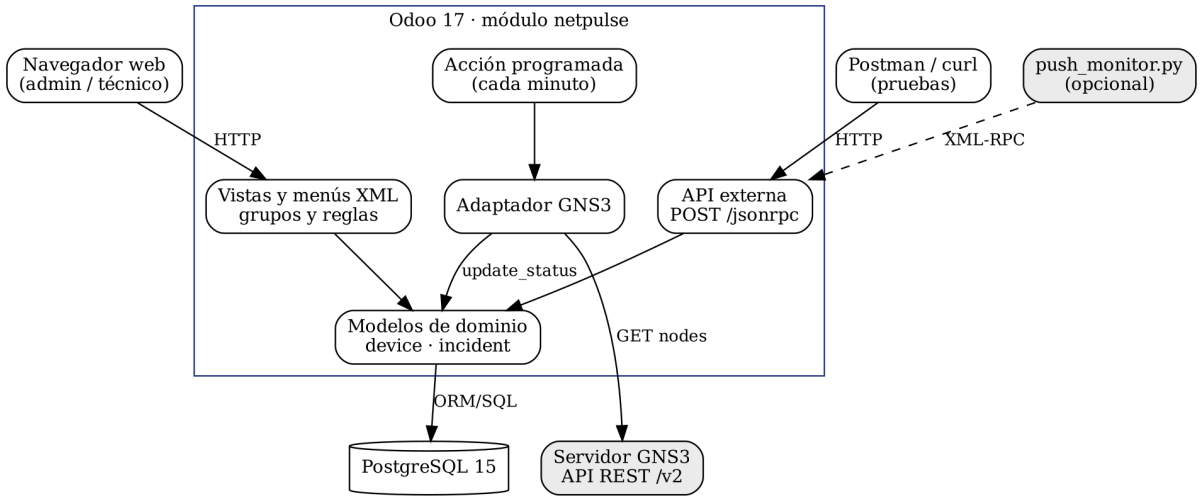


Diagrama de componentes: clientes, backend (Odoo), base de datos e integraciones externas. Todo corre en Docker Compose (Odoo 17 y PostgreSQL 15).

Parte	Estilo	Justificación
Cliente web	MVC del framework Odoo (vistas XML y cliente OWL), diseño responsivo	Un único cliente cubre administrador y técnico, también desde el celular; evita mantener dos front-ends.
Backend	MVC de Odoo (modelos con ORM, vistas XML, acciones) y un adaptador (puertos y adaptadores) solo para la integración con GNS3	El dominio no conoce GNS3: la integración vive tras `netpulse.gns3.adapter`. Autenticación, permisos y API externa los provee Odoo.
Datos	PostgreSQL 15 con el ORM de Odoo como repositorio	Transacciones, restricciones y reglas de acceso por registro.
Despliegue	Docker Compose (Odoo 17 + PostgreSQL 15), HTTP en local; Caddy o Nginx con HTTPS documentado para producción, sin desplegar	Entorno reproducible con un solo comando.

Reconocemos que Odoo no es un framework hexagonal puro: su ORM acopla dominio y persistencia. Lo aceptamos porque nos da seguridad, vistas y persistencia ya probadas, y aislamos con un adaptador la única integración externa riesgosa (GNS3).

3.2 Modelo de dominio (DDD)

Lenguaje ubicuo (glosario)

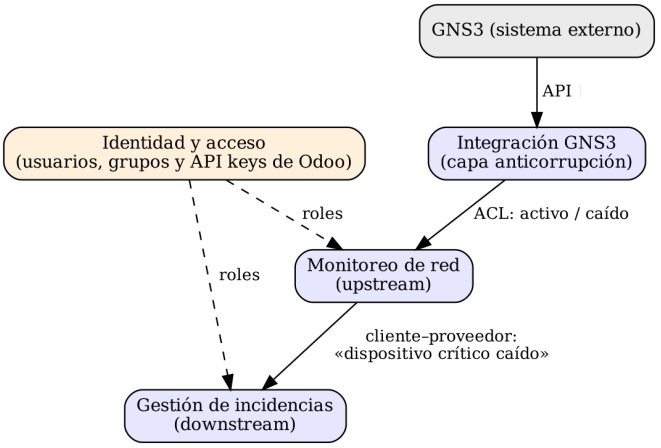
- **Dispositivo:** equipo de red monitoreado (router, switch u otro) registrado en el inventario.

- **Criticidad / dispositivo crítico:** importancia del dispositivo; la caída de uno crítico exige incidencia automática de prioridad Alta.
- **Nodo GNS3:** representación del dispositivo en el simulador; se enlaza por el ID de nodo GNS3.
- **Estado del dispositivo:** Activo, Caído o Desconocido (sin información fiable del monitoreo).
- **Topología:** vista de los dispositivos y su estado (verde, rojo, neutro).
- **Sincronización:** consulta periódica del estado de los nodos a GNS3 (ciclo de monitoreo).
- **Simulación de caída:** acción de demostración que fuerza un estado Caído o Activo sin GNS3.
- **Incidencia:** registro de un problema sobre un dispositivo, con código INC-nnnn.
- **Origen:** Automático (creada por el monitor) o Manual.
- **Prioridad:** urgencia de la incidencia: Baja, Media o Alta.
- **Causa raíz:** explicación breve del origen (p. ej. «SW2 offline»).
- **Ciclo de vida:** Pendiente → En proceso → Resuelta, sin saltos.
- **Asignación:** vínculo entre una incidencia y el técnico responsable.
- **Actividad pendiente:** aviso que recibe el técnico al ser asignado.
- **Administrador / Técnico:** los dos roles del sistema (grupos de Odoo).
- **API key:** credencial de Odoo, con hash, usada para llamar a la API externa.

Bounded contexts y context map

Se identifican cuatro contextos delimitados: **Monitoreo de red** (dispositivos, estados y topología), **Gestión de incidencias** (incidencias, asignación y ciclo de vida), **Identidad y acceso** (usuarios, grupos y API keys, provistos por Odoo) e **Integración GNS3** (traducción del sistema externo).

Monitoreo es upstream de Gestión de incidencias (relación cliente–proveedor): le comunica el hecho «dispositivo crítico caído». La Integración GNS3 es una capa anticorrupción que entrega a Monitoreo solo «activo / caído». Identidad y acceso es un contexto que Odoo ofrece a ambos.



Context map de NetPulse.

Agregados

1. Incidencia (agregado principal). Raíz: Incidencia. Entidades hijas: la actividad pendiente de asignación y los mensajes de seguimiento (historial). Value objects: código (INC-nnnn), prioridad, causa raíz y estado. Invariantes:

- Solo se permite Pendiente → En proceso → Resuelta y Resuelta es final.
- Hay a lo sumo una incidencia abierta por dispositivo.
- Una incidencia En proceso o Resuelta tiene técnico asignado.
- La incidencia automática de un dispositivo crítico nace con prioridad Alta y causa raíz.
- Solo el administrador cambia técnico y prioridad.
- El técnico solo ve y atiende las incidencias que tiene asignadas.

2. Dispositivo. Raíz: Dispositivo. Value objects: estado, criticidad e ID de nodo GNS3. Invariantes:

- Nombre único.
- Un nodo GNS3 pertenece a un solo dispositivo.
- Con el monitoreo, nunca se marca Caído sin evidencia: sin respuesta de GNS3 el estado es Desconocido (la simulación es la excepción explícita).

3.3 Diagrama de clases y SOLID

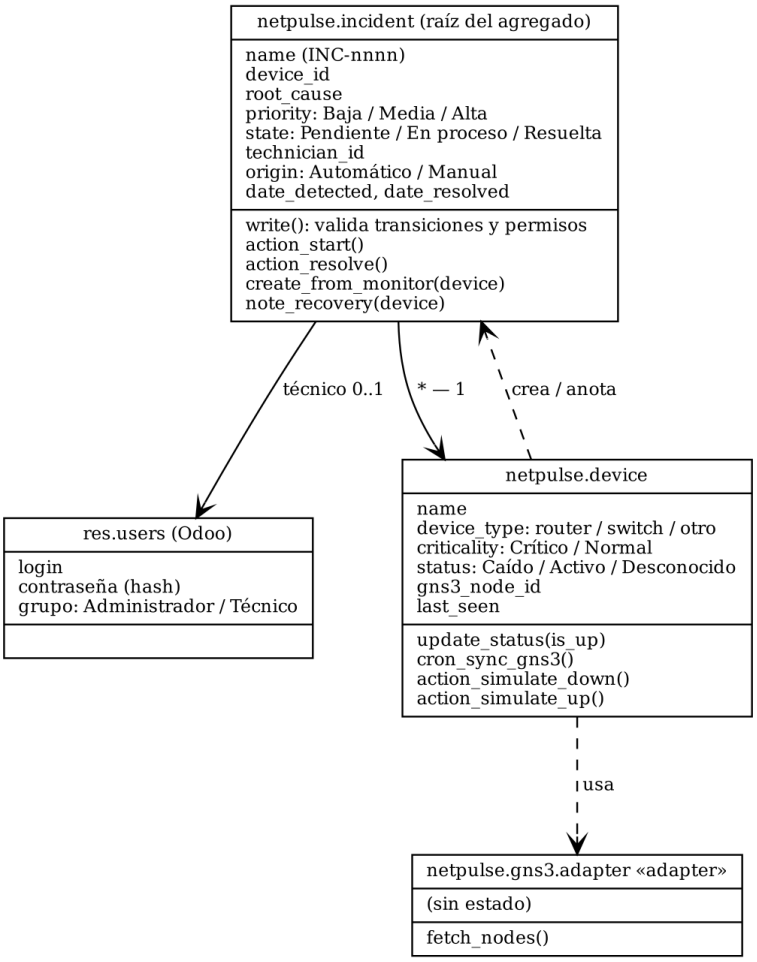


Diagrama de clases del dominio y la capa de aplicación. Las líneas continuas son asociaciones y las punteadas dependencias. Los casos de uso se materializan como métodos de los modelos de Odoo y la acción programada.

	Dónde	**Evidencia**
S	Incidencia / Dispositivo / Gns3Adapter	Cada uno cambia por un solo motivo: ciclo de vida de la incidencia, estado del dispositivo, traducción de GNS3.
O	Fuentes de estado	Una fuente nueva llama a `update_status` sin modificar las reglas de incidencias (el script `gns3_push_monitor.py` ya lo hace).
L	Fuentes de estado	El ciclo de GNS3, el script externo y la simulación entregan el mismo contrato: (dispositivo, está activo).
I	Gns3Adapter	Contrato mínimo: `fetch_nodes()`; `Dispositivo.update_status(is_up)` es el único punto de entrada de estados.
D	Dispositivo.cron_sync_gns3 → adaptador	La acción programada no conoce HTTP ni GNS3: obtiene los estados del adaptador, que puede reemplazarse o simularse en pruebas.

3.4 Patrones de diseño (GoF)

Patrón	Problema que resuelve	Clases que participan	SOLID
Adapter	La API de GNS3 no coincide con lo que necesita el dominio.	Target: `update_status` del dispositivo. Adapter: `netpulse.gns3.adapter`. Adaptee: API REST de GNS3.	D, O, S
State	La incidencia admite acciones distintas según su etapa.	Contexto: Incidencia. Estados: Pendiente, En proceso, Resuelta. Se implementa como tabla de transiciones permitidas en `write()`.	O, L
Observer (del framework)	Al asignar una incidencia, el técnico debe enterarse sin que el modelo conozca el canal de aviso.	Sujeto: Incidencia. Observador: actividad pendiente del técnico, creada con el mecanismo de actividades de Odoo desde `write()`.	O, D

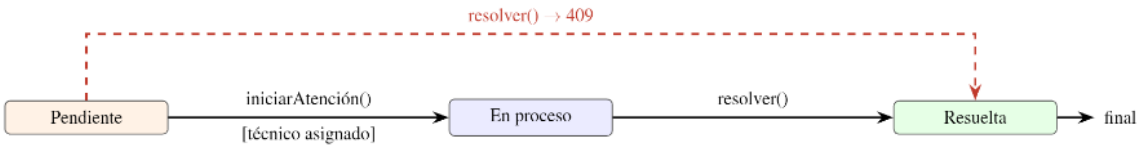


Figura 7: Ciclo de vida de la incidencia (patrón State). La transición directa Pendiente → Resuelta está prohibida.

Ciclo de vida de la incidencia (patrón State). La transición directa Pendiente → Resuelta está prohibida.

3.5 Atributos de calidad

E1 — Seguridad (autorización)

Fuente	Usuario técnico autenticado.
Estímulo	Intenta cambiar la prioridad o reasignar una incidencia, o eliminar un dispositivo.
Artefacto	Modelos `netpulse.incident` y `netpulse.device` (permisos por grupo, reglas de registro y validación en `write()`).
Entorno	Operación normal.
Respuesta	Rechaza con `AccessError` (≈403) y no modifica datos.
Medida	100 % de los intentos rechazados y 0 cambios persistidos; verificable con la colección de Postman y las pruebas del módulo.
Táctica	Autorizar actores (permisos por grupo + reglas de registro) y limitar el acceso.

E2 — Disponibilidad (falla de tercero)

Fuente	Servidor GNS3.
Estímulo	Deja de responder (tiempo de espera agotado).
Artefacto	Acción programada de monitoreo y adaptador GNS3.
Entorno	Operación normal.

Respuesta	Marca los dispositivos asociados a un nodo como Desconocido, no crea incidencias, registra el evento en el log y reintenta en el siguiente ciclo.
Medida	0 incidencias falsas; Odoo sigue atendiendo peticiones; lectura recuperada en ≤ 1 ciclo (1 min) al volver GNS3.
Táctica	Detección de fallas (timeout), degradación controlada, reintento.

E3 — Modificabilidad

Fuente	Desarrollador.
Estímulo	Debe integrar otro monitor (p. ej. Zabbix) como fuente de estados.
Artefacto	Capa de integración.
Entorno	Tiempo de desarrollo.
Respuesta	Se escribe un adaptador nuevo que entrega (dispositivo, estado) a `update_status`, o un script externo como `gns3_push_monitor.py`.
Medida	Meta de diseño: ≤ 1 día-persona y 0 reglas de incidencias modificadas.
Táctica	Encapsular con un adaptador y mantener un contrato estable.

E4 — Rendimiento

Fuente	Administrador.
Estímulo	Abre el dashboard o la lista de dispositivos con 200 dispositivos.
Artefacto	Vistas de dashboard y lista de dispositivos.
Entorno	Carga normal (hasta 10 usuarios simultáneos).
Respuesta	Muestra los dispositivos con su estado, con la paginación y agrupación de Odoo.
Medida	Meta de diseño, aún no medida: dashboard en menos de 2 s; se verifica en el Prototipo 2.
Táctica	Paginación nativa, agrupación en una sola consulta e índices en campos de búsqueda (nodo GNS3, estado de la incidencia).

4. API segura (parte implementada)

Odoo 17 no ofrece una API REST por recurso, expone una API externa JSON-RPC (`POST /jsonrpc`) sobre su ORM. Todas las operaciones usan ese único endpoint; lo que cambia es el servicio, el método y el modelo. Esto condiciona varios puntos de la guía, que se adaptan como se explica a continuación.

4.1 Especificación OpenAPI

`docs/openapi.yaml` (OpenAPI 3.0.3) documenta las operaciones del MVP sobre `/jsonrpc` (y `GET /web/health`) con esquemas, 9 ejemplos de petición, ejemplos de respuesta y de error, y el esquema de seguridad `bearerAuth` (API key de Odoo). Se validó con `openapi-spec-validator` y puede abrirse en https://editor.swagger.io. Como todas

las operaciones comparten un solo endpoint, se documentan como ejemplos nombrados de una misma operación.

Operación	Servicio / método / modelo	Rol
Iniciar sesión	`common.authenticate`	Cualquiera
Listar y leer dispositivos	`execute_kw` · `search_read` · `netpulse.device`	Administrador, técnico
Crear, modificar y eliminar dispositivos	`execute_kw` · `create`, `write`, `unlink` · `netpulse.device`	Administrador
Listar y leer incidencias	`execute_kw` · `search_read` · `netpulse.incident`	Administrador (todas), técnico (las suyas)
Crear incidencia	`execute_kw` · `create` · `netpulse.incident`	Administrador; técnico (queda a su nombre)
Asignar técnico o cambiar prioridad	`execute_kw` · `write` · `netpulse.incident`	Administrador
Iniciar atención y resolver	`execute_kw` · `write` del estado · `netpulse.incident`	Administrador, técnico (las suyas)
Eliminar incidencia	`execute_kw` · `unlink` · `netpulse.incident`	Administrador

Códigos de respuesta. Odoo responde con HTTP 200 incluso cuando hay un error; el error viaja en el cuerpo (`error.data.name`). Las equivalencias que usamos son: `AccessDenied` ≈ 401 (no autenticado), `AccessError` ≈ 403 (sin permiso) y `ValidationError` ≈ 400 (datos inválidos o transición no permitida).

4.2 Backend y organización por capas

Los dos recursos con CRUD son dispositivos (`netpulse.device`) e incidencias (`netpulse.incident`); el ORM de Odoo entrega las cuatro operaciones por la interfaz y por la API. La organización sigue la arquitectura de la Sección 3:

Capa	Dónde vive
Vistas y acciones	`addons/netpulse/views/*.xml` y `security/`: vistas, menús, grupos y permisos.
Casos de uso / servicios	Métodos de los modelos y acción programada de sincronización (`create_from_monitor`, `update_status`, `action_start`, `action_resolve`, `cron_sync_gns3`).
Dominio	Modelos Odoo `netpulse.device` y `netpulse.incident` con sus restricciones e invariantes (`models/`).
Repositorios	ORM de Odoo sobre PostgreSQL 15.
Integración externa	`netpulse.gns3.adapter` (adaptador hacia GNS3) y `scripts/gns3_push_monitor.py` (alternativa por API externa).

4.3 Autenticación

La guía pide JWT con registro e inicio de sesión. En Odoo la autenticación es nativa y no se implementó JWT ni un endpoint de registro; el equivalente es:

- **Inicio de sesión:** en la interfaz web con usuario y contraseña (sesión de Odoo). Por API, `common.authenticate` valida usuario y API key y devuelve el `uid`; con credenciales inválidas devuelve `false`.
- **Registro:** las cuentas las crea el administrador desde Ajustes → Usuarios (HU-02); no hay registro público.
- **Contraseñas con hash:** Odoo nunca guarda texto plano; usa PBKDF2-SHA512 con sal. Lo mismo aplica a las API keys.
- **API keys:** cada usuario genera la suya en Preferencias → Seguridad de la cuenta; se usa en lugar de la contraseña en las llamadas JSON-RPC. Opcionalmente puede añadirse el módulo OCA `auth_jwt`.

4.4 Autorización por roles (401 vs. 403)

Operación	Sin credencial válida	Técnico	Administrador
Listar / consultar dispositivos	`AccessDenied` (≈401)	Permitido	Permitido
Crear / modificar / eliminar dispositivos	`AccessDenied` (≈401)	`AccessError` (≈403)	Permitido
Ver incidencias	`AccessDenied` (≈401)	Solo las suyas	Todas
Cambiar prioridad o reasignar técnico	`AccessDenied` (≈401)	`AccessError` (≈403)	Permitido
Iniciar atención / resolver	`AccessDenied` (≈401)	Permitido (las suyas)	Permitido (requiere técnico asignado)

Hay defensa en profundidad: los permisos por grupo del modelo y las reglas de acceso por registro (el técnico no lee incidencias ajenas) y, además, validaciones en `write()` del servidor que impiden que un técnico cambie técnico o prioridad aunque llame directamente a la API.

4.5 Buenas prácticas de seguridad

- **Secretos fuera del código:** las credenciales de PostgreSQL y de GNS3 se leen de variables de entorno (`.env`, ignorado por Git); el repositorio solo contiene `.env.example` con valores de ejemplo.
- **Gestión de bases de datos deshabilitada:** `list_db = False` y `dbfilter` fijo en `config/odoo.conf`. `proxy_mode` queda en `False` en local y debe activarse detrás de un proxy HTTPS.

- **Validación de entrada:** campos obligatorios, restricciones de unicidad, validación de transiciones y de incidencias duplicadas; los datos inválidos devuelven `ValidationError`.
- **Errores:** Odoo devuelve un formato único (`error.data.name` y `error.data.message`). **Limitación:** el error JSON-RPC de Odoo incluye también un campo `data.debug` con la traza del servidor; no se suprimió en este prototipo (ver riesgos).
- **CORS:** no se configuró, porque el cliente web lo sirve el mismo Odoo (mismo origen) y no hay otros clientes en navegador.
- **Cuenta inicial:** la base creada por línea de comandos trae el usuario `admin` con la contraseña `admin`; el README indica cambiarla en el primer ingreso.

4.6 HTTPS

En desarrollo se usa HTTP en `localhost`. Para producción se documenta poner Odoo detrás de Caddy o Nginx, que obtiene y renueva un certificado Let's Encrypt y redirige HTTP a HTTPS (`reverse_proxy odoo:8069`), con `proxy_mode = True` para que Odoo confíe en los encabezados del proxy. El backend **no está desplegado** todavía.

4.7 Colección de Postman

`postman/NetPulse.postman_collection.json` (17 peticiones, todas con pruebas automáticas) y `postman/NetPulse.postman_environment.json` (sin llaves). Las pruebas se ejecutan completas y en orden con el Collection Runner; los identificadores se guardan solos en variables de la colección y las API keys se escriben en Current value del environment, sin subirlas al repositorio. Como Odoo responde HTTP 200, las pruebas verifican el cuerpo (`result` o `error`).

Peticiones	Qué cubren
01–03	Autenticación: login de administrador y de técnico, credenciales inválidas (`authenticate` devuelve `false`)
04–06	CRUD de dispositivos: crear, leer y actualizar
07–09	Incidencias: crear asignada al técnico, leer y pasar a En proceso (transición válida)
10–15	Seguridad y validación: API key inválida (`AccessDenied`), técnico elimina dispositivo (`AccessError`), técnico cambia prioridad (`AccessError`), técnico lee su incidencia, incidencia sin dispositivo (datos inválidos) y transición no permitida (`ValidationError`)
16–17	Limpieza: eliminar la incidencia y el dispositivo de prueba

5. Decisiones, riesgos y próximos pasos

5.1 Decisiones clave

- **Odoo 17 como backend:** acelera seguridad, persistencia y vistas; a cambio, el dominio queda acoplado al ORM.

- **Adaptador para GNS3 y modo simulación:** la demostración no depende de la red del laboratorio.
- **Estado Desconocido en vez de Caído** cuando no hay datos, para no crear tickets falsos.
- **API externa de Odoo (JSON-RPC) con API keys**, en lugar de una capa REST/JWT propia: evita duplicar lo que Odoo ya provee; JWT y REST quedan como mejora.
- **Permisos validados en el servidor** (`write()`, permisos por grupo y reglas de registro), no solo ocultando botones en la interfaz.

5.2 Riesgos

Riesgo	Mitigación
Odoo no ofrece JWT, OpenAPI generado ni códigos HTTP 401/403 propios	`openapi.yaml` escrito a mano, equivalencias documentadas (`AccessDenied` ≈ 401, `AccessError` ≈ 403); evaluar los módulos OCA `auth_jwt` y `base_rest` en el Prototipo 2.
Los errores JSON-RPC incluyen la traza del servidor (`data.debug`)	No exponer Odoo a Internet sin proxy; filtrar el campo en el proxy o usar una capa REST en el Prototipo 2.
GNS3 corre en el computador de un integrante	Modo simulación (HU-11) y script `gns3_push_monitor.py` para Odoo en la nube.
Fuga de secretos por el historial de Git	`.gitignore`, `.env.example`, revisión del historial y rotación de claves si algo se sube.
Contraseña inicial `admin` de la base creada por línea de comandos	Cambiarla en el primer ingreso (indicado en el README).
Rendimiento sin medir	Los escenarios E3 y E4 son metas de diseño; se miden con 200 dispositivos en el Prototipo 2.

5.3 Próximos pasos hacia el Prototipo 2

Métricas históricas de disponibilidad, modificación del frontend para que sea más fiel a los wireframes iniciales, funcionalidad completa de GNS3 sin depender del modo simulación, capa REST con JWT (si los módulos OCA lo permiten) y despliegue gratuito solo por HTTPS.

Anexo A. Cobertura de la guía del Prototipo 1

Sección de la guía	Estado	Nota
1. Requisitos (a–f)	Cumple	Secciones 1.1 a 1.5.
2.a Wireframes web y móvil	Parcial	Web en Figma (algunas pantallas); móvil cubierto por el diseño responsivo de Odoo, sin wireframes móviles.
2.b Flujo de navegación	Cumple	Sección 2.2.
2.c Modelo de cliente	Cumple	Web responsiva; justificado en 2.3.

Sección de la guía	Estado	Nota
3.a Arquitectura	Adaptado	MVC de Odoo más adaptador; no hay cliente móvil separado.
3.b a 3.e Dominio, clases, patrones, calidad	Cumple	E3 y E4 son metas de diseño aún no medidas.
4.a OpenAPI	Adaptado	Documenta la API JSON-RPC de Odoo, escrita a mano.
4.b CRUD en dos recursos	Cumple	Vía ORM; capas propias de Odoo.
4.c JWT, registro y hash	Adaptado	Sin JWT ni endpoint de registro; sesión y API keys con hash.
4.d Roles, 401 y 403	Adaptado	`AccessDenied` y `AccessError`, con HTTP 200.
4.e Buenas prácticas	Parcial	Secretos en variables de entorno y validación; el error incluye `data.debug`; sin CORS propio.
4.f HTTPS	Parcial	Configuración descrita; sin desplegar.
4.g Postman	Implementado	17 peticiones con pruebas automáticas.
4.h API externa (bonificación)	No reclamada	A consultar con el profesor.
5. Repositorio y trabajo en equipo	En curso	README, `.gitignore` y `.env.example` listos; ramas y pull requests dependen del trabajo del equipo.